Exercise 04

Task 1

Code is in `src/task1`, needs the zenoh-c and zenoh-cpp api to compile. As annotated in the code, the logging and serialization helpers were written by the ai, the rest by me.

Task 2

First I just started the 3 nodes and it elected A as the leader. I got the following example output:

A:

```
[21:05:17.013] [nodeA | term 9 | LEADER] [HEARTBEAT] sending heartbeat (311 sent)
[21:05:19.016] [nodeA | term 9 | LEADER] [HEARTBEAT] sending heartbeat (321 sent)
```

B:

```
[21:05:17.013] [nodeB | term 9 | FOLLOWER] [HEARTBEAT] received from
leader nodeA (311 total)
[21:05:19.016] [nodeB | term 9 | FOLLOWER] [HEARTBEAT] received from
leader nodeA (321 total)
```

C:

```
[21:05:16.212] [nodeC | term 9 | FOLLOWER] [HEARTBEAT] received from
leader nodeA (171 total)
[21:05:18.215] [nodeC | term 9 | FOLLOWER] [HEARTBEAT] received from
leader nodeA (181 total)
```

After I stopped A, C was the first to time out and it had this output:

C:

```
[21:05:20.876] [nodeC | term 9 | FOLLOWER] [TIMEOUT] no heartbeat
received, starting election
[21:05:20.877] [nodeC | term 10 | CANDIDATE] [VOTE] voted for self,
sending vote request
[21:05:20.879] [nodeC | term 10 | CANDIDATE] [VOTE] received vote (2
of 2 nodes, need more than 1)
[21:05:20.879] [nodeC | term 10 | LEADER] [STATE] CANDIDATE ->
LEADER with 2 votes
[21:05:20.927] [nodeC | term 10 | LEADER] [HEARTBEAT] sending
heartbeat (1 sent)
```

B:

```
[21:05:20.879] [nodeB | term 10 | FOLLOWER] [VOTE] granted vote to
nodeC
[21:05:21.930] [nodeB | term 10 | FOLLOWER] [HEARTBEAT] received
```

```
from leader nodeC (331 total)
```

After then starting A again it gets prints this: A:

```
[21:05:35.759] [nodeA | term 9 | FOLLOWER] [HEARTBEAT] from nodeC
has newer term 10, updating term
[21:05:35.760] [nodeA | term 10 | FOLLOWER] [HEARTBEAT] received
from leader nodeC (1 total)
```

SIDENOTE:

When looking at the terms you might ask: "Why is it already at 10?". This is because currently, if there is only 1 node, the election majority formula doesn't quite work here. It would be an easy fix but depening on the semantics, you might want, that it waits for at least one other node, before actually starting.

Task 3

- Why are randomized election timeouts critical for preventing perpetual split votes? If every node gets the timeout at the exact same time, and will start an election at the exact same time. This means every node will always vote for himself and a consensus is never reached.
- What happens if two candidates receive an equal number of votes in a cluster, and how does the system recover? I implemented it, that for a canditate to win it needs the majority, so this means that the election will fail for both of them after a random timeout. Then each node will restart an election.
- How does comparing currentTerm prevent an isolated or delayed node from disrupting the active cluster with stale messages? A node with a lower current term will just always be ignored, so older messages can never disrupt current elections.
- Is leader election alone sufficient to guarantee linearizable consistency in a distributed database? Explain briefly. No, since the leader could potentially disconnect from the other nodes. These other nodes start a new leader election, while the stale leader has no information about this. If a new request comes in to the stale leader, this could lead to inconsistencies. So it needs an additional mechanism of confirming, that the leader is still connected to everything/all other nodes agree that he is the leader.